

(Optional) WiFi-based Communication

Introduction

WiFi+Packer Sender is an alternative to UART+TeraTerm. You need to do only either of them, though if you wish to do both, that is fine too!

The B-L475E-IOT01A board comes with an ISM43362-M3G-L44 WiFi module from Inventek Systems. The default firmware of the module is meant to communicate with the host processor (Cortex M4) through the SPI interface.

The IoT platform example uses **Thingsboard**  (<https://thingsboard.io/>). Lately, the Thingsboard demo server has been a bit slow at times - your mileage may vary, but we still recommend trying it out.

Networking Basics

To understand more about any term mentioned below, just Google. The Internet had more information on networking than most other topics. After all, the internet is the most widely used application of networking :smile:.

Each computer on a network is identified (at the 'network layer') using an IP address.

A connection is identified using 5 parameters - the transport layer protocol, source IP address, source port, destination IP address, and destination port. A port is just a logical interface on a computer. In order for an application ('client') on your computer (source) to connect to another computer (destination), you need to know the IP address of the destination computer, as well as the port number. An application ('server') on the destination computer will be listening to a particular port, and once a connection is made, the two applications can talk to each other. The source port number for the connection will be assigned by your OS and the source IP address is your computer's IP address - these typically need not be specified explicitly. The transport layer protocol is typically TCP or UDP - TCP is used for applications requiring data reliability. Applications, where late data is as good as lost data (e.g.: live streaming), use UDP.

Application layer protocols such as HTTP(S), FTP, etc. ride on such a transport layer connection.

When you enter google.com in your browser, the process that happens is

1. A DNS lookup is performed by sending a request to a DNS server (maintained by your ISP, or manually).
2. Once the IP address of google.com is obtained, the browser initiates a TCP connection to <destination>:80, where 80 is the destination port number, to which Google's web server application is listening. The protocol used is HTTP, which rides on the transport layer protocol TCP.
3. Once a connection is established, the two applications can send data to each other through the connection. HTTP is a request-response-based protocol, so the server will only respond to requests. In HTTP, the two most popular requests to request/send information are GET and POST. These requests and associated payloads are sent in packets. For example, the browser sends a request for the index page first up, and the server responds with the index page (HTML), which the browser then renders.

The way we send information to an IoT server (telemetry) is quite similar, especially when using HTTP. To send data, we do a GET or POST with an appropriately formatted payload. The format differs from server to server, and you should consult the documentation of the IoT platform. The response from the IoT server is typically some form of acknowledgment, such as a status code of 200 if the server considers the request it received from you as valid. An application layer protocol like MQTT, but HTTP is arguably easier to get started without the need for a library.

Most IoT platforms support visualization (dashboards) and processing/analytics of telemetry data, as well as storage. This can be based on data processing, user input from a separate app, etc.

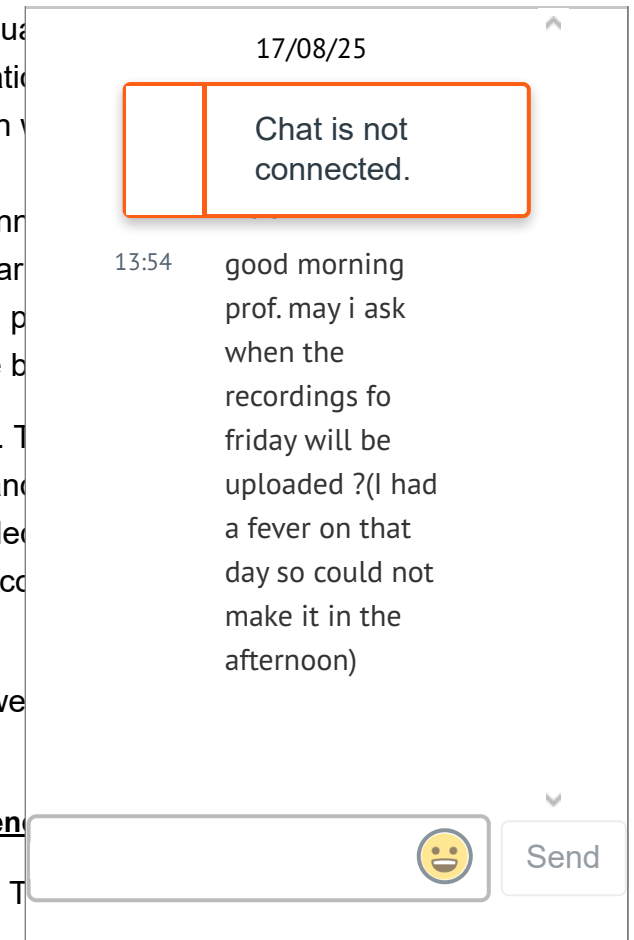
The example program follows **Thingsboard telemetry HTTP API** [\(https://thingsboard.io/docs/reference/telemetry-api/\)](https://thingsboard.io/docs/reference/telemetry-api/).

For local communication without any external server (using Packet Sender, see below), we use a 'raw' TCP layer protocol. Packet Sender can act as the 'server', and your STM32 application acts as the 'client'.

WiFi Connection

Both the board and your laptop will need to connect to the same network (LAN). You have 3 options to choose from

1. A wireless router (e.g., router you have at home) - not an option if you plan to include it in your demo
2. WiFi hotspot on your phone.
3. 'Mobile hotspot' on your Windows laptop to convert the laptop into a hotspot to which your board can connect. Please see [this link](https://support.microsoft.com/en-us/windows/use-your-pc-as-a-mobile-hotspot-c89b0fad-72d5-41e8-f7ea-406ad9036b85) <https://support.microsoft.com/en-us/windows/use-your-pc-as-a-mobile-hotspot-c89b0fad-72d5-41e8-f7ea-406ad9036b85> on how your Windows laptop can be converted into a hotspot. Only **2.4 GHz** networks are supported by the WiFi chip on the board, so make sure that you choose



2.4 GHz for 'Network band' in step 3 in this link rather than the default, which is 'Any available'. You might also want to turn off the 'Power Saving' option under 'Mobile Hotspot' to avoid the hotspot turning off when you are disconnected for a while.

You **cannot connect to NUS WiFi** using the board (the security used by NUS - WPA2 Enterprise, is not supported by the WiFi chip on the board).

Finding the IP Address

You need to find the IP address of your laptop network adapter. You can use Start Menu > cmd > ipconfig /all. It will be in the format 192.168:x:y, where x and y are numbers between 0 and 255.

Look up the 'IPv4 Address' of the network adapter that connects your laptop to your router - likely your wireless card, which Windows will label as 'Wireless LAN adapter Wi-Fi'.

If the board is connected via Windows hotspot, there will be another IP address which is likely 192.168.137.1, which can also be used instead of the address found in the previous step. You can see this as 'IPv4 Address' under 'Wireless LAN adapter Wi-Fi Local Area Connection* 1', which will have 'Microsoft WiFi Direct Virtual Adapter' in the description.

Packet Sender

Download and install Packet Sender from <https://packetsender.com/download#show>  (<https://packetsender.com/download#show>). Allow Firewall-related prompts in Windows.

Packet Sender can be used as a console for TCP/IP, the same way we use TeraTerm for UART (Serial). Packet Sender can be used as a server as well as a client. We will be using it as a server, which the client running on STM32 Chip will be connecting to.

Go to File > Settings. Give the settings as below. The port number should be between 1024 and 65535. If you are happy with 2028 and are not facing any issues, you do not need to change it. Note that some ports could be used by other programs (e.g.: GDB server for debugging), so if you face issues, try another port.

Packet Sender Settings

Network
Display
Smart Responses

Basic

☐ Enable UDP Servers
 UDP Server Ports (comma-separated, 0 for random)

☒ Enable TCP Servers
 TCP Server Port (comma-separated, 0 for random)

☐ Enable SSL/TLS Servers
 SSL Server Port (comma-separated, 0 for random)

Set the server ports to 0 if you want to run multiple Packet Sender instances.

☐ Send a basic response with macro support

Response Data

ASCII

HEX

Additional UDP/TCP/SSL Settings

☒ IPv4 Only
 ☐ IPv6 Only
 ☐ Bind This:

Cancel Resending after # packets

☐ Attempt Receive Before Send

☐ 500 ms delay after connect (slow devices)

☒ Persistent TCP Connections

☐ Enable Ctrl (or Cmd)+ E to translate ASCII field to EBCDIC

☐ Resolve DNS during input (old method)

☒ Ignore SSL Errors (Wrong host, expired, self-signed, etc)

☒ SSL Server may use an internal snake oil cert

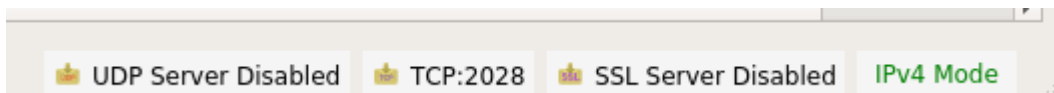
SSL CA Certificates

SSL Local Certificate

SSL Private Key

[Online Documentation](#)

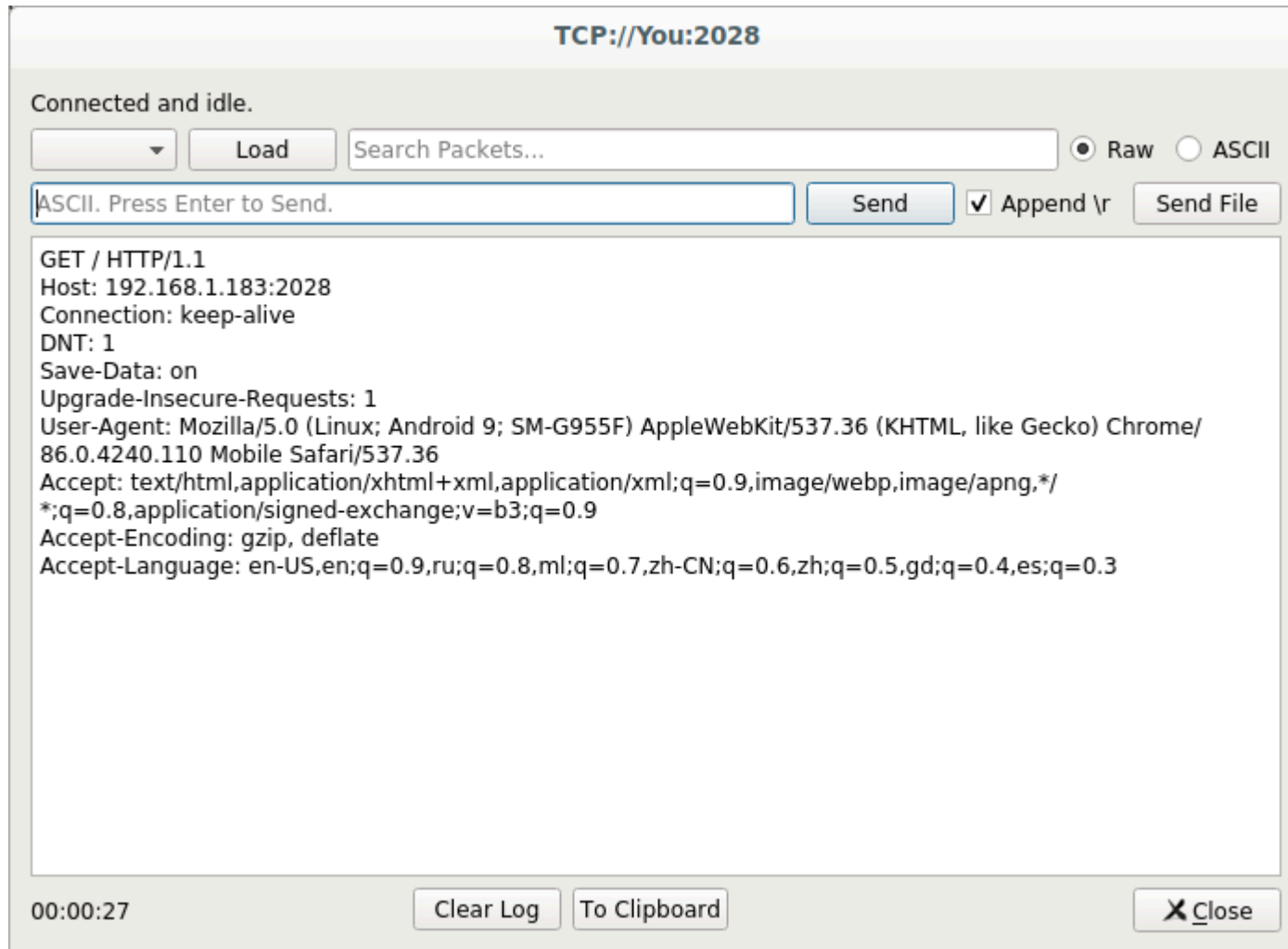
There is no configuration required in the main window itself, but make sure that the bottom part of the main window shows as follows.



You will also be able to see the IP addresses the Packet Sender will be listening to in the title bar as shown below. One of them will be the IPv4 address of the wireless adapter your laptop uses to connect to the router/access point. If there is only one IPv4 address, you have not turned on the Windows hotspot, which is fine if you are connecting your board via a router/phone hotspot. If you have turned on Windows hotspot, you will be able to see 192.168.137.1 (likely) also (not the case in the screenshot below).



To test connectivity to Packet Sender, you can use another 'device' such as a phone or another laptop to simulate the connection your board will be making later on. Connect this 'device' to the router/phone hotspot/Windows hotspot as applicable. Open up a web browser in this 'device', and type in 192.168.x.y:2028 in the address bar where x and y correspond to the IP address of your laptop which you found in the previous section. You should be able to see the Packet Sender window popping up as shown below (assuming Packet Sender was open in the first place). If it does, great, your board should also be able to talk to the Packet Sender!



The rest of the stuff in this section is just for fun and is optional (not fundamentally important). As long as you are able to see the request from the browser displayed in Packet Sender (as in the screenshot above - the exact message could vary depending on the browser you use), it is good enough and shows that Packet Sender is able to listen to the specific port and receive packets from the network. If you don't see the screenshot above, chances are that Packet Sender can't receive messages from the network.

The above message is an HTTP request sent by your browser to the 'server', which in this case is Packet Sender. You can make Packer Sender give a response by copy-pasting the content below into the 'ASCII. Press Enter to Send' box and click the Send button or press the Enter key.

HTTP/1.0 200 OK

Content-Type: text/html; charset=UTF-8

Content-Length: 178

Connection: close

\n

<!doctype html>

<html>

<head>

<title>It Works!</title>

</head>

<body>

<p style="color:blue;font-size:54px;"> EE2028 is cool!</p> </body></html>

You will be able to see your browser displaying a simple webpage, 'served' by Packet Sender. The browser may send another HTTP request (for favicon), which you can ignore.

14:51 M 🔑

🔔 🔊 VoLTE 📶 43% 🔋

⚠️ 192.168.137.1:2028

93



EE2028 is cool!

Modifying the STM32 Program

You need to have the following files in your project, in a location in which the compiler will look for source files.

Headers : es_wifi_conf.h, es_wifi_io.h, es_wifi.h, wifi.h

Sources : es_wifi_io.c, es_wifi.c, wifi.c

Here is a .zip archive with the above files - **Wifi(optional).zip** (<https://canvas.nus.edu.sg/courses/78652/files/7719416?wrap=1>) 
(https://canvas.nus.edu.sg/courses/78652/files/7719416/download?download_frd=1)

If you want to be very safe, you can copy all the .h files under Core>Inc, and all .c files under Core>Src. You can then delete the Drivers>BSP>Components>es_wifi folder. Note that you can right-click and select copy in your OS's file explorer (windows explorer, for e.g.), and paste into the folder inside your folder in STM32CubeIDE by right-clicking and selecting Paste.

Insert the following code into Core>Src>stm32l4xx_it.c (or replace the existing EXTI1_IRQHandler, if it exists)

```
void EXTI1_IRQHandler(void)

{

    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_1);

}
```

A sample main program is shown in "[Wifi_sample.rtf \(https://canvas.nus.edu.sg/courses/78652/files/7719376?wrap=1\)](https://canvas.nus.edu.sg/courses/78652/files/7719376?wrap=1)  [\(https://canvas.nus.edu.sg/courses/78652/files/7719376/download?download_frd=1\)](https://canvas.nus.edu.sg/courses/78652/files/7719376/download?download_frd=1)". You need to change the program below to enter the correct values for the various definitions/constants. You will, at the very least, need to change WiFi_SSID, WiFi_password, ipaddr (of your laptop/PC) if using Packet Sender. If using Thingsboard IoT, make sure you enter your API key in line 58 of the code below.

When you run the program, if the execution stops somewhere for unexplained reasons (this happens when a breakpoint does not get removed properly), you can just click Resume (F8 key) to let it continue.

Make sure you have HAL_GPIO_EXTI_Callback() and SPI3_IRQHandler() in your main.c, as well as EXTI1_IRQHandler() in stm32l4xx_it.c when you eventually merge it with your existing main program.

Using Packet Sender

Comment out the USING_IOT_SERVER definition.

The IP address of your computer can change (via DHCP) every time your computer reconnects to the router/hotspot (it can change in a few hours even without a reconnection, though this is unlikely). Make sure you are using the latest IP address.

Packet Sender should be running **before** you run the STM32 program. Otherwise, it will try to make a connection to a socket (IP address + port number) nobody is listening to which won't succeed.

If everything is done right, once your board starts sending info, you will be able to see a window as shown below pop up, showing you the info sent by your board.

Any info you type in the box having "ASCII. Press Enter to Send" will be sent (not as you type, but only when you press Enter) to the board, and your program can receive it through `WIFI_ReceiveData()`.



Close the window that popped up (but not the Packet Sender application itself) **before** you start a fresh debugging session. Else, the new window will be behind the previous window (i.e., in the background) and you won't see the updated values sent (until you close the foreground window).

It is not that difficult to write a program using **Python sockets**  (<https://docs.python.org/3/howto/sockets.html>) to do the same job as Packet Sender, but gives you the possibility to do visualization (**matplotlib**  (<https://matplotlib.org/>), should be handy), advanced data processing (e.g., machine learning), automated replies to the board, etc.

Using IoT Platform

Attempt an IoT server connection only after you have managed to get everything to work using Packet Sender. If you are using Thingsboard, no modification to the program is required other than entering the **API key**, which is unique for your account and device. You can Google to figure out how to create an account, device, and dashboard in Thingsboard. You can also refer to the short video below which demonstrates this process. However, note the observation at the beginning of this page.

Thingsboard allows you to make your dashboard public, allowing anyone on the internet with the dashboard URL to see the dashboard live!

If you are using an IoT platform other than Thingsboard, you will have to modify the program as appropriate to follow the API of the IoT platform you are using. You will need to enter the correct domain name, destination port number (protocol-dependent), as well as API key, in addition to making appropriate API-specific modifications to the data format. Please refer to documentation and examples for the specific server you are using.

It is entirely possible to send information to both Packet Sender and IoT server. You will have to `WIFI_OpenClientConnection()` twice, once for each connection. The first parameter is the connection index, which needs to be different for each connection. In `WIFI_SendData()` and `WIFI_ReceiveData()`, the connection index determines which server connection you are talking over, i.e., which server you are talking to. The information sent need not be the same. For example, you might want to send the messages using formats specified in the project specification when you send to Packet Sender, whereas messages to an IoT server should conform to the API of that IoT server.

Truly Standalone System

To make your board truly standalone and untethered to the PC/Laptop, you can simply power it from a power bank, and everything should still work without any connection to PC/Laptop. This should be done after clicking debug* at least once, which will cause your program to be written

to flash.

You can also program the board using the .bin file instead of using the debugger. You can find the .bin file in the Debug* folder of your project. The board will appear as a USB drive on your computer with the name DIS_L4IOT. Just copy the .bin file into the drive.

*It is better that you program it using the Release version. Change the build version to Release and build it. You can either write the program using the debugger as usual (debuggability in release mode is very limited and should not be used for debugging) or by copying the .bin file over. You will find the .bin file in the Release folder of your project. The release version will have a smaller binary size and will also run faster.

